

CROC DRIVE : A Motor- and Power-Control Peripheral for Croc SoC

VLSI 2 Final Project Report – vlsi2_12fs26

Chelsea Lai

Rishabh Barola

chelai@student.ethz.ch

rbarola@student.ethz.ch

June 30, 2026

Abstract

This report describes CROC DRIVE , our extension of the Croc RISC-V SoC with a small motor- and power-control peripheral. The motivation was practical: generating PWM or decoding encoder transitions purely in software quickly turns the processor into a timing loop instead of a controller. We therefore moved the time-critical parts, PWM edge generation and quadrature decoding, into a memory-mapped AMCP block while keeping the control algorithm in firmware.

AMCP is integrated as an OBI subordinate in the Croc user domain and reuses the existing GPIO interface through an alternate-function mux, so the CVE2 core, pad interface, and basic Croc test infrastructure remain compatible with the reference design. The design was verified with standalone RTL tests, full-system Verilator firmware tests, PWM/QEN benchmarks, and closed-loop-style motor/buck demonstrations using testbench plant models. The final backend implementation meets setup and hold timing, passes LVS, and has clean GDS-level DRC except for the known SRAM macro off-grid/min-grid markers. In the reported VCD workloads, AMCP stress consumes 53.2 mW versus 52.3 mW for the software-PWM stress case, corresponding to about 1.7 % full-chip overhead.

1 Introduction and Contributions

Modern microcontrollers often sit between software and a physical system. The processor computes decisions, but the actual interaction with the outside world happens through accurately timed pins: switching a motor driver, driving a power stage, sampling feedback, or counting sensor transitions. Croc provides a compact open-source RISC-V SoC platform for such extensions, with a CVE2-based processor, memory, standard peripherals, and a user domain connected through the OBI interconnect [1, 2, 3]. The question addressed in this project is: which parts of a small closed-loop control system should remain in software, and which parts should be moved into simple deterministic hardware?

Two operations are especially poorly suited to pure GPIO bit-banging. The first is pulse-width modulation (PWM). A PWM signal is a periodic digital waveform whose duty cycle controls the average command seen by an external circuit. This is widely used because a digital chip can generate only high/low levels, while an external motor driver, LED driver, low-pass filter, or buck-converter power stage interprets the duty cycle as an analog-like control input [4]. If PWM is generated in software, the CPU must repeatedly toggle a GPIO pin at precise cycle times, and any interrupt, branch, memory stall, or bus access can disturb the waveform.

The second operation is quadrature encoder decoding. A quadrature encoder provides two phase-shifted digital signals, A and B, whose transition sequence encodes both displacement and direction. Many encoders also provide a Z or index pulse once per mechanical revolution, giving a known reference position [5]. If these signals are decoded by software polling, fast or noisy transitions can be missed. A small hardware decoder can instead accumulate position, direction, index, and error information continuously, while software reads the accumulated state when needed.

This project extends Croc with CROC DRIVE , a compact Advanced Motor-Control Peripheral (AMCP) placed in the user domain. The design is deliberately modest: it does not replace the processor with a fixed-function controller. Instead, it moves only the timing-critical I/O functions into hardware. The CPU still configures the peripheral, runs benchmark and control firmware, and updates control parameters. The AMCP autonomously generates PWM edges and decodes quadrature transitions, exposing the resulting state through memory-mapped registers.

The AMCP is integrated as an OBI subordinate rather than by modifying the CVE2 core. This preserves the existing Croc programming model: software accesses AMCP using normal RISC-V loads and stores, the JTAG/UART simulation flow remains unchanged, and the user-domain address decoder routes accesses to the new peripheral. The peripheral is register-based because PWM and QEN require only configuration bits, counters, status flags, and small signed state variables; no local memory is required inside AMCP.

The chip-level interface is also kept conservative. Instead of adding new top-level pins, CROCDRIVE reuses the existing GPIO interface through an alternate-function mux. By default, all pins behave as normal Croc GPIO. When `AMCP_PINMUX[0]` is set, GPIO0 is driven by the hardware PWM output. QEN feedback can come from synchronized GPIO inputs or from a software-controlled simulation-input register for deterministic tests. This avoids pad-ring changes and reduces backend risk while still exposing the new real-time I/O function at the chip boundary.

The closed-loop demonstrations validate the digital control path rather than claiming a complete mixed-signal power or motor driver. In the motor-oriented tests, PWM acts as the actuator signal and QEN provides position-style feedback. In the buck-converter demonstration, the testbench observes the real PWM output, simulates an averaged buck-converter plant, and drives an ADC-like feedback value into GPIO inputs. Thus the chip implements the digital controller interface and timing-critical I/O, while the analog plant and ADC are modeled only in simulation.

In addition to AMCP, two system-level changes were made to support the larger verification and control workloads. First, the SRAM capacity was increased from two 2 KiB banks to two 8 KiB banks. This was possible because the AMCP logic occupied only a small fraction of the available floorplan area, while the larger C firmware tests were otherwise memory-constrained. Second, the CVE2 core configuration was changed from no RV32M multiplier to the fast RV32M multiplier. This supports integer arithmetic in benchmark and closed-loop control firmware. This is not a custom accelerator contribution, but a practical system configuration choice enabled by the available area. The main contributions are:

- AMCP integration as a memory-mapped OBI subordinate in the Croc user domain.
- Hardware PWM generation, QEN decoding, and GPIO0 alternate-function routing without changing the pad interface.
- A preserved user ROM at `0x2000_0000`, enlarged SRAM configuration, and RV32MFast-enabled firmware platform.
- Verification using standalone RTL tests, full-system firmware simulations, PWM/QEN benchmarks, closed-loop demonstrations, backend implementation, power analysis, and IR-drop analysis.

2 Architecture and RTL Design

2.1 System-Level Architecture

Figure 1 shows the modified Croc architecture. The baseline Croc SoC already contains the CVE2 core, debug logic, SRAM banks, UART, GPIO, timer, and a user domain connected through the main OBI crossbar. Our design keeps this structure intact and adds two user-domain subordinates: a read-only user ROM and AMCP.

The user ROM is mapped at `0x2000_0000`. It stores the required null-terminated group-identification string as little-endian 32-bit words. The AMCP is mapped at `0x2000_1000`. Both blocks occupy separate 4 KiB windows, which keeps the address map explicit and avoids overlapping functionality. Any user-domain address not covered by these two regions is routed to an OBI error subordinate.

The AMCP is subordinate-only. It does not initiate OBI transactions and therefore does not use the user-domain manager port. The unused manager request is tied to zero. Template ports such as `ref_clk_i` remain in the user-domain interface for compatibility with the baseline Croc structure, but AMCP itself uses only `clk_i`. Unused template signals are explicitly terminated where needed to document the design intent and avoid tool warnings about intentionally unused ports.

2.2 Memory Map and System Configuration

Table 1 summarizes the relevant memory map after our modifications. The SRAM map was updated because each SRAM bank was enlarged from 512 32-bit words to 2048 32-bit words. Therefore each bank grows from 2 KiB to 8 KiB, and the two-bank system grows from 4 KiB to 16 KiB. The main crossbar address map in `croc_pkg.sv` was updated accordingly.

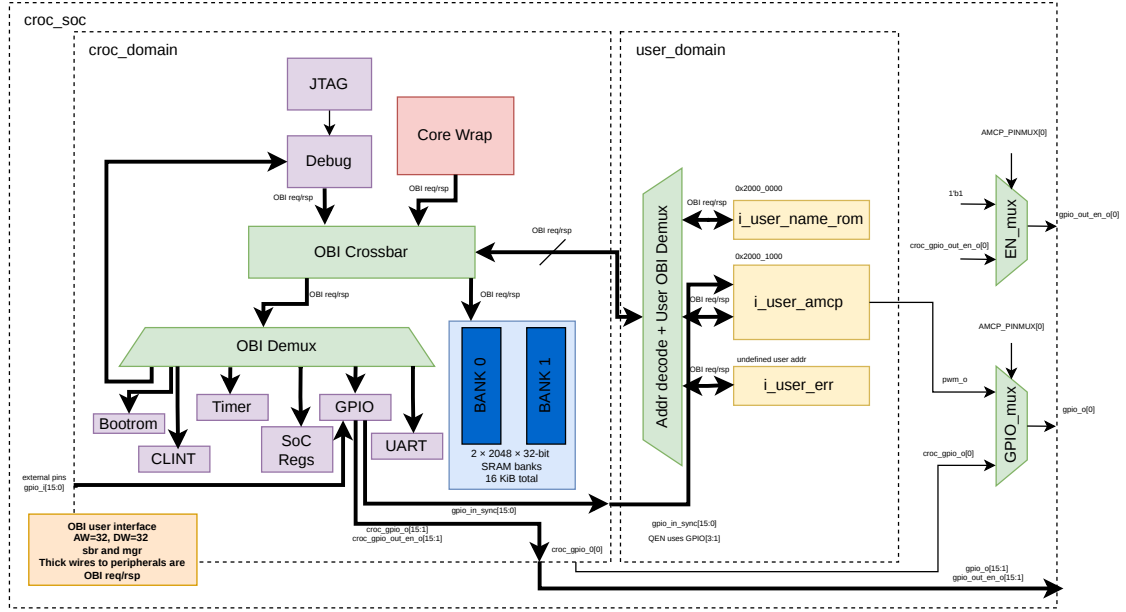


Figure 1: System-level integration of CROC DRIVE . AMCP is connected as an OBI subordinate in the Croc user domain. PWM reaches the chip boundary through a GPIO0 alternate-function mux, while QEN can use synchronized GPIO inputs or software-injected test inputs.

Table 1: Relevant memory map of the modified design. End addresses are exclusive.

Start	End	Region	Description
0x1000_0000	0x1000_2000	SRAM bank 0	8 KiB SRAM bank, 2048 32-bit words
0x1000_2000	0x1000_4000	SRAM bank 1	8 KiB SRAM bank, 2048 32-bit words
0x2000_0000	0x2000_1000	User ROM	Null-terminated group-identification string
0x2000_1000	0x2000_2000	AMCP	PWM, QEN, status, debug, and pinmux registers

The user-domain address decoder was therefore changed from one placeholder range into two concrete subordinates plus a default error target. This keeps the required ROM at the user base address while making AMCP independently decodable.

2.3 AMCP Module Structure

Figure 2 shows the internal structure of `user_amcp`. The wrapper contains four main parts: a one-cycle OBI request/response interface, a register decoder, a PWM core, and a QEN core. The PWM and QEN cores are intentionally kept free of OBI logic. They receive clean control signals from the AMCP register file and return datapath/status signals to the register interface. This separation keeps the real-time datapaths reusable and makes the bus-facing logic easier to verify.

AMCP accepts every OBI request immediately:

```
obi_rsp_o.gnt = obi_req_i.req.
```

The request fields `req`, `we`, `addr`, `wdata`, `be`, and `aid` are captured into internal registers. One cycle later, `rvalid` is asserted and the response data, error flag, and response ID are returned. The response ID is copied from the captured request ID so that responses remain associated with their original requests.

Register selection is based on the 32-bit word offset inside the AMCP 4 KiB window:

```
reg_word_offset = addr_q[7:2].
```

Bits `addr_q[1:0]` are not needed because all software-visible registers are 32-bit aligned. This gives compact decoding for offsets such as `0x000`, `0x004`, `0x008`, and so on.

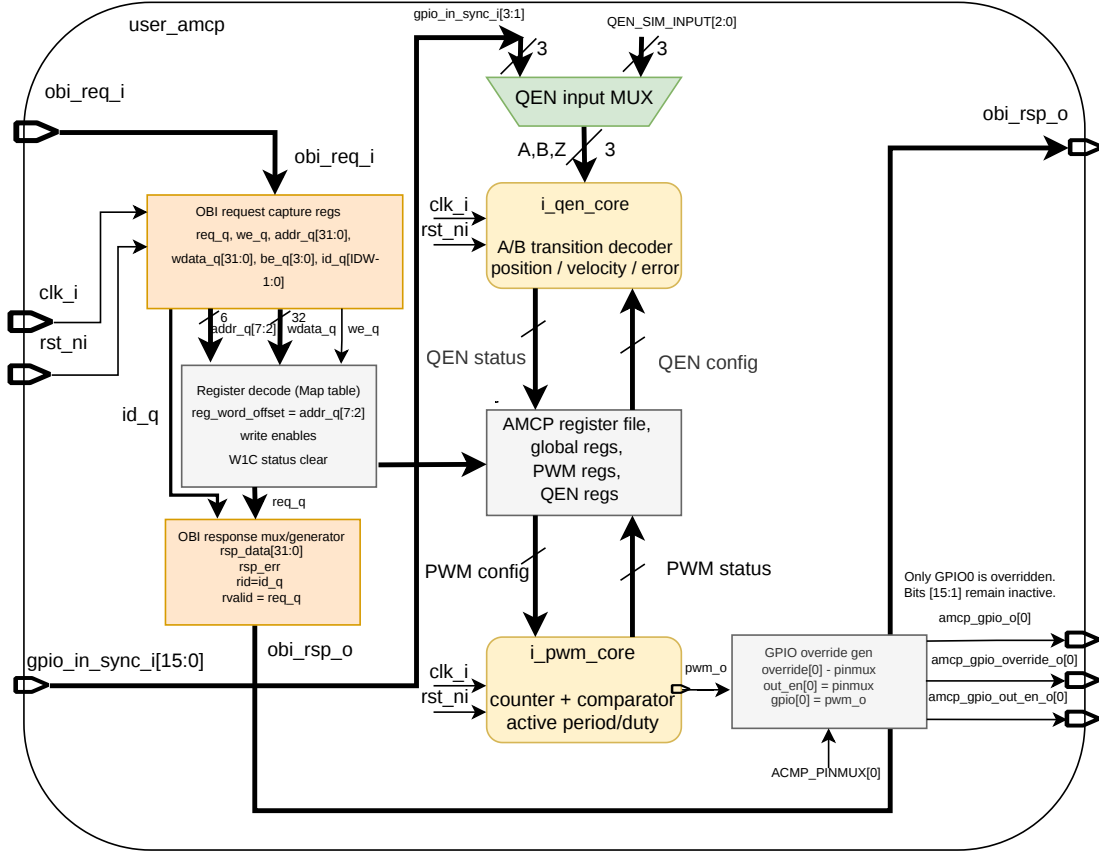


Figure 2: Internal structure of `user_amcp`. The OBI request is captured, decoded into software-visible registers, and used to configure independent PWM and QEN cores. The PWM output is routed to GPIO0 through the AMCP GPIO override signals.

2.4 Register Access Policy

Table 2 lists the AMCP registers. All software-visible registers are 32-bit OBI registers, even when the internal datapath uses fewer bits. This keeps the C software interface simple and aligned with the Croc OBI data width.

The access and reset columns in the register map are derived directly from the RTL behavior. Read-only registers return an OBI error on writes. Read/write registers update stored state on writes and return the stored value on reads. Write-one-to-clear status registers contain flags that hardware sets and software clears by writing ones. The reset value is the value software reads immediately after reset. For stored registers this comes from the reset branch of the corresponding `always_ff` block; for constants such as `AMCP_ID`, it comes from a hardwired `localparam`.

The register map intentionally contains several status and readback registers. These are not redundant: they make the hardware observable and robust. PWM has separate configuration and active readback registers because software writes to `PWM_PERIOD` and `PWM_DUTY` do not necessarily take effect immediately during an active PWM cycle. Reading `PWM_PERIOD_ACTIVE` and `PWM_DUTY_ACTIVE` tells firmware and tests which values are actually driving the waveform. `PWM_STATUS` makes one-cycle events, such as period completion or invalid zero-period operation, visible to software even if software polls more slowly than the PWM clock.

For QEN, the position, velocity, delta, index, and error registers expose accumulated encoder state. This avoids requiring the CPU to react to every A/B transition. The control register includes both persistent mode bits and write-one pulse command bits. In particular, `QEN_CTRL[1]` clears position-related state and `QEN_CTRL[2]` clears the error counter; these bits are command pulses and are not stored. `QEN_SIM_INPUT` exists only to make deterministic full-system tests possible without an external encoder model.

Table 2: AMCP register map relative to base address 0x2000_1000.

Offset	Name	Access	Reset	Description
0x000	AMCP_ID	R	0x414D4350	Hardwired ASCII “AMCP” identifier
0x004	AMCP_VERSION	R	0x1	Hardwired AMCP version
0x008	AMCP_PINMUX	R/W	0x0	Bit 0 selects PWM alternate function on GPIO0
0x00C	AMCP_STATUS	R	0x0	Reserved global status
0x010	PWM_CTRL	R/W	0x0	Bit 0 enable, bit 1 polarity
0x014	PWM_PERIOD	R/W	0x0	Configured PWM period, lower 16 bits used
0x018	PWM_DUTY	R/W	0x0	Configured PWM high time, lower 16 bits used
0x01C	PWM_COUNTER	R	0x0	Current PWM counter value
0x020	PWM_STATUS	R/W1C	0x0	Sticky period-done and zero-period flags
0x024	PWM_DUTY_ACTIVE	R	0x0	Duty value currently used by the core
0x028	PWM_PERIOD_ACTIVE	R	0x0	Period value currently used by the core
0x040	QEN_CTRL	R/W+W1P	0x10	Enable, clear pulses, index reset, simulation-input mode
0x044	QEN_STATUS	R	0x2	Enable, direction, index, error, invalid, velocity-valid flags
0x048	QEN_POSITION	R	0x0	Signed accumulated position
0x04C	QEN_VELOCITY	R	0x0	Position change over last sample window
0x050	QEN_DELTA	R	0x0	Last sampled position delta
0x054	QEN_INDEX_POSITION	R	0x0	Position captured on last Z-index pulse
0x058	QEN_ERROR_COUNT	R	0x0	Count of invalid A/B transitions
0x05C	QEN_SAMPLE_PERIOD	R/W	0x10	Velocity sample interval in clock cycles
0x060	QEN_SIM_INPUT	R/W	0x0	Software-injected {Z,B,A} input bits
0x064	QEN_DEBUG	R	0x0	Current selected QEN inputs and A/B state

2.5 PWM Core

The PWM generator is implemented in `pwm_core.sv`. It produces a periodic digital waveform whose duty cycle is controlled by software. This is useful for control applications because the CPU can write a desired period and duty value once, after which the hardware maintains the output timing autonomously. The PWM core itself contains no OBI logic; it receives clean control signals from AMCP and returns counter, active-value, and status information.

The PWM datapath in Figure 3 consists of a 16-bit counter, active period/duty registers, duty-clamp logic, a wrap detector, and an output comparator. Although the software-visible registers are 32 bits wide, only the lower 16 bits of `PWM_PERIOD` and `PWM_DUTY` are used by the core. This keeps the counter and comparator small while providing sufficient resolution for the demonstrated workloads.

The duty value is clamped before use. If the configured period is zero, the duty is forced to zero. If the configured duty is greater than or equal to the configured period, it is clamped to the period value. This makes edge cases well-defined instead of relying on software to avoid invalid values.

The PWM core is implemented as a counter-based waveform generator. Software writes the desired period and duty cycle into configuration registers. Internally, the core keeps separate active period and duty values, which are the values currently being used to generate the waveform. This separation is needed because software may update the configuration while a PWM cycle is already in progress. If the counter immediately used the new values, the current pulse could become shorter or longer than intended. Therefore, new configuration values are loaded only when PWM is disabled, when recovering from an invalid zero-period configuration, or at the end of a PWM period. This keeps each generated PWM cycle well defined.

The PWM status and readback registers are included to make the hardware observable from software. The counter register allows firmware and tests to inspect the current phase of the PWM cycle. The active period and active duty registers show which values are actually being used by the waveform generator, which may differ temporarily from the newly written configuration registers until the next period boundary. The status register stores short hardware events, such as period completion or zero-period operation, so that software can observe them even if it polls more slowly than the PWM clock.

The polarity bit provides a simple way to select whether the active part of the PWM cycle is high or low. This is useful because different external drivers may interpret an active-high or active-low control signal differently. However, when PWM is disabled or the active period is zero, the output is forced low. This gives a safe inactive output state independent of the polarity setting.

For an active period value N ,

$$f_{\text{PWM}} = \frac{f_{\text{clk}}}{N}, \quad \Delta D = \frac{1}{N}.$$

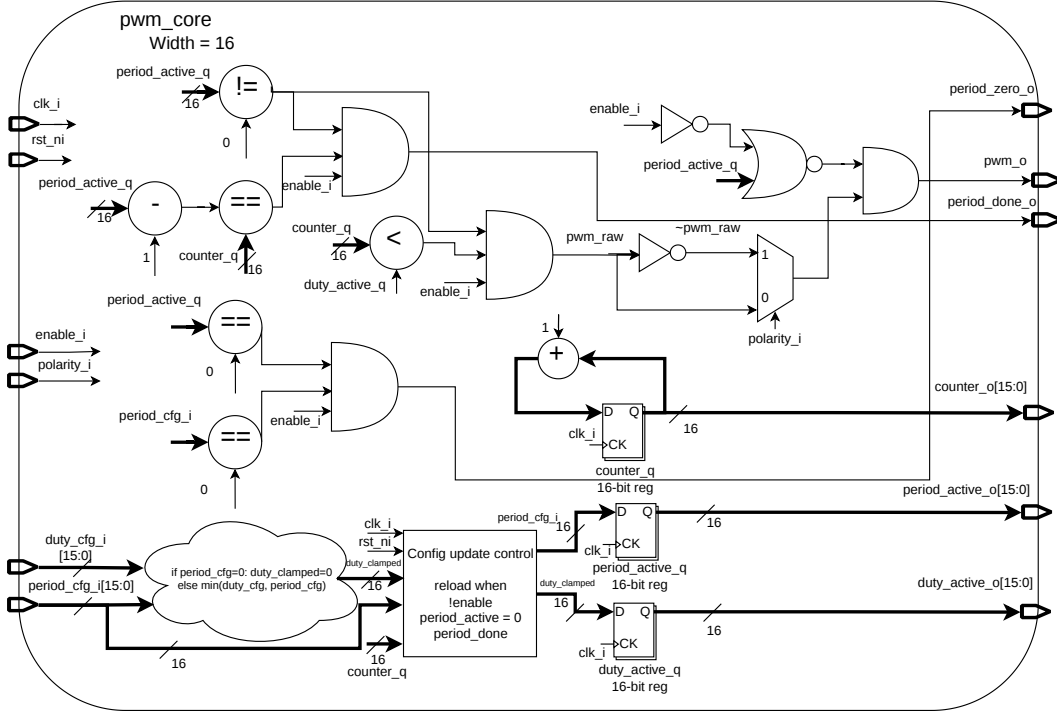


Figure 3: PWM core datapath. Software writes configuration values through AMCP, while the core uses separate active period and duty registers, a counter, a wrap detector, and a comparator to generate a stable PWM output.

The 1 MHz, 1 % waveform used in the buck-converter demo is therefore one selected operating point with $N = 100$ at 100 MHz; it is not a fixed architectural limit.

The output is generated by

$$\text{pwm_raw} = \text{enable} \wedge (\text{period_active} \neq 0) \wedge (\text{counter} < \text{duty_active}).$$

The polarity bit optionally inverts the active waveform. If PWM is disabled or the active period is zero, the output is forced low. The block does not use a large explicit FSM; it is a sequential counter datapath with priority cases for reset, disabled/preload, zero-period recovery, period-boundary update, and normal counting.

2.6 Quadrature Encoder Core

The quadrature encoder interface is implemented in `qen_core.sv`. It converts encoder A/B input transitions into software-readable motion state. Instead of requiring the CPU to poll GPIO pins and decode every transition in software, the hardware keeps track of position, direction, invalid transitions, index pulses, and position change over a programmable sample window. AMCP provides the enable signal, clear commands, index-reset configuration, sample period, and selected A/B/Z input source. The QEN core returns signed position, velocity-like delta, direction, index, and error information through the AMCP register map.

A quadrature encoder provides two phase-shifted digital inputs, A and B. The sequence of A/B states encodes both displacement and direction. The QEN core stores the previous A/B state in `prev_ab_q` and compares it with the current state `curr_ab = {b_i, a_i}` on every clock cycle. The state encoding is $\{B, A\}$. Valid forward transitions increment the signed position counter, valid reverse transitions decrement it, unchanged states leave the counter unchanged, and illegal two-bit jumps increment the error counter. The valid forward sequence is $00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00$, while the valid reverse sequence is $00 \rightarrow 10 \rightarrow 11 \rightarrow 01 \rightarrow 00$; repeated states leave the position unchanged, and other two-bit jumps increment the error counter.

Figure 4 shows the QEN datapath. The transition decoder is combinational: it classifies $\{\text{prev_ab_q}, \text{curr_ab}\}$ into forward, reverse, unchanged, or invalid. The resulting signed step is added to the current position only for valid transitions. The sequential update logic then applies a fixed priority: reset, software clear, disabled state, first-enable initialization, transition update, index capture, and sample-window update.

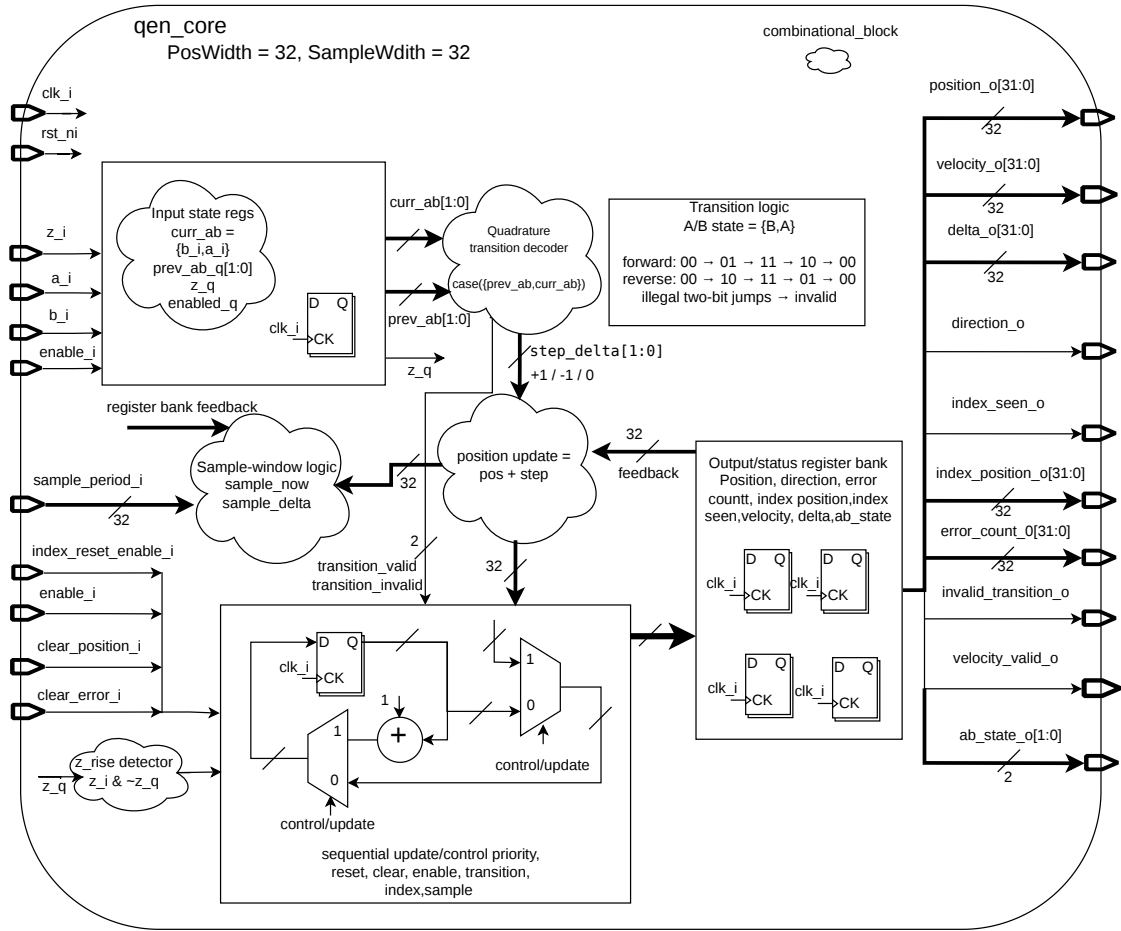


Figure 4: QEN core datapath. The previous A/B state and current A/B input classify each quadrature transition. The core accumulates signed position, captures index position, counts invalid transitions, and estimates velocity from position differences over a programmable sample window.

The Z input is the encoder index signal. In many quadrature encoders, this signal produces one pulse per revolution at a known mechanical position. The QEN core detects a rising edge of Z by comparing the current value z_i with the previous sampled value z_q . When such an edge occurs, the current position is copied into $index_position_q$ and the $index_seen$ flag is set. If index-reset mode is enabled, the position and velocity-sampling state are also reset after this capture. This allows software to use the index pulse as a reference point while still knowing at which count the pulse occurred.

Velocity is reported as a digital displacement over a programmable sample window, not as a physical-unit speed. The core stores a previous sample position and increments a sample counter. When the sample counter reaches QEN_SAMPLE_PERIOD , it computes

$$sample_delta = position_after_step - sample_position_q.$$

This value is written to both $QEN_VELOCITY$ and QEN_DELTA , and $velocity_valid$ pulses for one cycle. If software writes zero to the sample-period register, the hardware internally uses one cycle as the effective minimum period, avoiding a zero-length sampling interval. The QEN register set separates configuration, observation, and debugging. Control bits select enable, index-reset mode, and GPIO versus simulation-input source; clear-position and clear-error are write-one pulse commands rather than stored configuration bits. Position, direction, index, velocity, and error registers expose accumulated encoder state, while QEN_SIM_INPUT supports deterministic firmware tests without an external encoder model. In normal GPIO mode, A, B, and Z come from synchronized GPIO inputs GPIO1, GPIO2, and GPIO3.

2.7 GPIO Alternate-Function Routing

The PWM output reaches the chip boundary through a mux in `croc_soc.sv`. The baseline Croc GPIO peripheral still drives internal signals `croc_gpio_o` and `croc_gpio_out_en_o`. AMCP provides three corresponding vectors: `amcp_gpio_o`, `amcp_gpio_out_en_o`, and `amcp_gpio_override_o`. The final top-level GPIO signals are selected bitwise:

$$\begin{aligned}\text{gpio_o} &= (\text{amcp_gpio_override} \wedge \text{amcp_gpio_o}) \vee (\neg \text{amcp_gpio_override} \wedge \text{croc_gpio_o}), \\ \text{gpio_out_en_o} &= (\text{amcp_gpio_override} \wedge \text{amcp_gpio_out_en}) \vee (\neg \text{amcp_gpio_override} \wedge \text{croc_gpio_out_en}).\end{aligned}$$

Only GPIO0 is overridden in the implemented design. AMCP drives `amcp_gpio_o[0]` with the PWM output and asserts `amcp_gpio_override_o[0]` and `amcp_gpio_out_en_o[0]` when `AMCP_PINMUX[0]` is set. All other AMCP GPIO override bits are tied to zero, so GPIO[15:1] remain under the baseline Croc GPIO peripheral. This approach exposes the PWM waveform without changing the top-level chip pin list or pad ring.

The Croc verification VIP was also extended to support external GPIO stimulus. The default behavior still loops selected GPIO outputs back to inputs for baseline GPIO tests. However, an external override mask can now replace selected GPIO inputs with testbench-driven values. This was needed for QEN tests and for the buck-converter demo, where the testbench plant model drives ADC-like feedback values into GPIO inputs.

3 RTL Implementation and Verification

The previous section described the AMCP architecture and the internal PWM/QEN datapaths. This section focuses on the implementation and verification methodology. The RTL implementation was kept modular: the bus-facing register logic is contained in `user_amcp.sv`, while `pwm_core.sv` and `qen_core.sv` are pure logic blocks without OBI-specific code. This allowed the cores to be tested independently before running full-system firmware simulations through Croc.

3.1 Implementation Summary

The main RTL changes are contained in `user_pkg.sv`, `user_domain.sv`, `user_amcp.sv`, `pwm_core.sv`, `qen_core.sv`, `croc_soc.sv`, and `croc_pkg.sv`. A separate `user_name_rom.sv` preserves the required group string at `0x2000_0000`. The full-system testbench `tb_croc_soc.sv` was extended with external GPIO stimulus, waveform markers, plant models, and VCD dump control.

3.2 Standalone RTL Verification

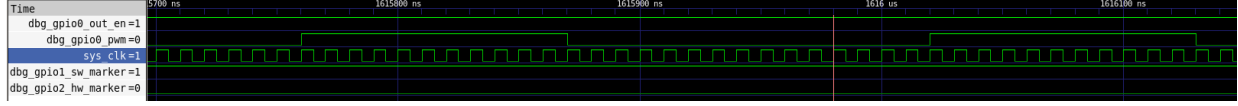
The PWM and QEN cores were first verified outside the full SoC using file-driven SystemVerilog testbenches and Python-generated golden responses. The PWM regression covered active-high and active-low operation, zero duty, full duty, duty larger than period, shortest valid period, zero-period behavior, and randomized period/duty/polarity cases. The QEN regression used directed cases followed by 10000 deterministic random vectors, for a total of 10042 vectors, and checked position, velocity, delta, direction, index state, invalid-transition behavior, error count, and A/B state. Additional directed velocity tests verified that `QEN_VELOCITY` and `QEN_DELTA` report signed displacement over the configured sample window.

3.3 Full-System Firmware Tests

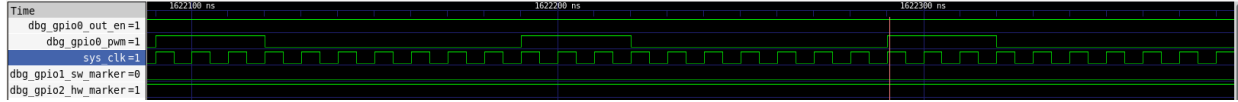
After standalone RTL verification, AMCP was tested inside the complete Croc system using C firmware loaded through the existing JTAG/VIP flow. The main tests were `test_pwm.c`, `test_qen.c`, `test_qen_gpio.c`, `benchmark_pwm.c`, `benchmark_qen.c`, and the closed-loop motor/buck demonstrations. Together, these verify the full path from RISC-V load/store instructions through the OBI interconnect to AMCP registers, PWM output routing, QEN operation with software-injected and externally driven A/B/Z stimuli, CPU-offload benchmarks, and firmware-controlled operation. Raw UART logs were decoded into cycle counts, PWM periods, QEN positions, error counters, and closed-loop values for reporting.

Table 3: PWM CPU-involvement benchmark. Hardware PWM has a fixed one-time setup cost; software PWM cost scales with the number of generated periods.

Periods	SW cycles	HW setup cycles	CPU reduction
10	1,643	13	126×
1,000	164,003	13	12,615×
10,000	1,640,003	13	126,154×



(a) Software bit-banded PWM phase.



(b) AMCP hardware PWM phase.

Figure 5: PWM waveform comparison in Verilator. The software waveform is limited by instruction/MMIO overhead, while AMCP produces the programmed counter-based waveform autonomously.

3.4 PWM Benchmark and Waveform Comparison

The PWM benchmark compares CPU involvement for software bit-banging versus AMCP hardware PWM. The software baseline toggles GPIO0 in a polling loop. The hardware case writes the AMCP configuration registers once, routes PWM to GPIO0, and then the PWM core continues autonomously. Table 3 reports the measured CPU cycles from the first benchmark. The hardware configuration cost is the number of `mcycle` cycles required to write the AMCP configuration registers once. It does not mean that the PWM period is 13 cycles. After this setup cost, the PWM core continues without further CPU intervention. The offload factor is computed as software PWM cycles divided by hardware configuration cycles.

This result does not mean that the PWM waveform is 126,154× faster. The correct interpretation is that the CPU involvement is much lower. Software PWM consumes CPU cycles continuously, while hardware PWM has a small fixed setup cost.

A second benchmark compares CPU availability over a fixed 100 000 cycle window. With hardware PWM, the programmed 10-cycle PWM period produces 10000 PWM periods in the window, while the CPU remains free to execute 511 iterations of a synthetic integer workload. In the software PWM case, the CPU spends the window toggling GPIO and waiting on cycle deadlines; it produces 3334 software PWM periods and completes no background-work iterations.

Figure 5 shows the corresponding waveform-level comparison. The hardware PWM produces the programmed 10-cycle period with 3 high cycles and 7 low cycles. The software bit-banded waveform, intended to produce the same nominal period, has an observed period of approximately 27 cycles because GPIO stores, cycle-counter reads, branches, and loop overhead take time. This illustrates the main advantage of moving PWM generation into hardware: the timing is *counter-based and deterministic*, and the CPU is released for other work.

The functional tests use the JTAG end-of-code status as the pass/fail criterion. Some UART-heavy verbose runs were repeated with a slower 20 MHz testbench clock to obtain readable console logs for decoding and report tables. The representative pass/fail simulations and the reported cycle-count benchmarks are based on architectural cycle counters and do not rely on UART timing for correctness.

3.5 QEN Verification and Benchmarking

The QEN verification was split into two parts. First, `test_qen.c` verifies the QEN register semantics using the software-controlled `QEN_SIM_INPUT` register. This isolates the QEN logic from external GPIO timing and checks that the CPU-visible registers behave correctly. Second, `test_qen_gpio.c` disables simulation-input mode and uses the Verilator testbench to drive GPIO1, GPIO2, and GPIO3 as external A/B/Z encoder inputs. This verifies the real chip-level feedback path through the GPIO interface.

Table 4 summarizes the external-GPIO QEN stress test. Each mode is selected by firmware, while the testbench drives a deterministic A/B/Z sequence. Firmware then reads `QEN_POSITION`, `QEN_ERROR_COUNT`, and `QEN_INDEX_POSITION`

Table 4: External-GPIO QEN verification results. Here QEN simulation-input mode is disabled, so A/B/Z are driven through GPIO1/GPIO2/GPIO3 by the testbench.

Mode	Exp. pos.	Obs. pos.	Exp. err.	Obs. err.	Exp. index	Obs. index	Result
Forward stress	400	400	0	0	–	–	PASS
Reverse stress	-160	-160	0	0	–	–	PASS
Mixed direction	80	80	0	0	–	–	PASS
Invalid burst	0	0	16	16	–	–	PASS
Index no-reset	60	60	0	0	40	40	PASS
Index reset	20	20	0	0	40	40	PASS

and compares the observed register values against the expected result. The test returns success only if all comparisons pass.

These cases cover signed accumulation, direction changes, invalid two-bit jumps, and index capture with and without position reset. The invalid-burst case confirms that illegal transitions increment the error counter without changing position.

The QEN benchmark compares software quadrature decoding against hardware QEN tracking for externally driven encoder transitions. The software path decodes every A/B transition in C, so its cost grows with the number of encoder edges. The hardware path configures QEN once, disables QEN_SIM_INPUT, and then lets the testbench drive the external GPIO encoder stream while the QEN hardware tracks the position autonomously. The CPU later reads the final position and error count.

Table 5: QEN software-decoding versus hardware-tracking benchmark. Hardware configuration cost is a one-time CPU cost; the QEN then tracks external GPIO transitions autonomously.

Transitions	SW cycles	SW cyc./trans.	HW config cyc.	Useful work	SW pos.	HW pos.	Errors
100	2,124	21	228	34	100	100	0/0
400	8,424	21	228	59	400	400	0/0
1600	33,624	21	228	157	1600	1600	0/0

The benchmark shows the intended offload behavior. Software decoding costs approximately 21 CPU cycles per transition, so its total cost scales linearly with encoder activity. In contrast, the hardware QEN configuration cost stays constant at 228 cycles for all tested transition counts. After configuration, QEN tracks the external GPIO A/B stream without requiring the CPU to decode every edge. The final hardware position matches the software golden result for all three cases, and both software and hardware report zero invalid-transition errors.

For the largest tested case, software decoding requires 33624 cycles, while hardware QEN setup requires 228 cycles. This corresponds to a CPU-involvement reduction of approximately

$$\frac{33624}{228} \approx 147.5 \times .$$

This does not mean that the encoder signal itself is faster. It means that continuous edge tracking has been moved from software into hardware, leaving the processor available for other work.

3.6 Closed-Loop Buck Demonstration

The buck demo verifies the digital control path. The testbench observes the real AMCP PWM output on GPIO0, measures the high count over each PWM period, updates an averaged buck-converter plant model, and drives a 10-bit ADC-like feedback value into GPIO inputs. Firmware reads this feedback, runs a fixed-point controller, and updates the PWM duty register. For $V_{in} = 1.2$ V and a 700 mV target, the controller settles around 58–59 high counts in a 100-cycle PWM period, corresponding to approximately 696–708 mV in the averaged plant model.

3.7 Limitations of the Verification

The verification strategy combines standalone cycle-accurate checks, full-system firmware tests, and benchmark workloads, but it is not a formal proof. The random tests are deterministic because they use fixed seeds, so failures are reproducible, but they do not cover every possible sequence. The software PWM baseline is a simple polling-based GPIO implementation; it is a fair direct comparison against bit-banging, but it is not an optimized timer-interrupt or

DMA-based implementation. The buck and motor plants are testbench models, not physical analog blocks. These limitations are acceptable for this project because the goal is to demonstrate correct integration of deterministic digital control I/O into Croc.

4 Physical Implementation

4.1 Backend Flow and Main Physical-Design Choices

The design was implemented using the course reference backend flow based on Yosys, OpenROAD, KLayout, and Calibre/KLayout verification. The final OpenROAD output set consists of `croc.v`, `croc_lvs.v`, `croc.def`, `croc.odg`, and `croc.sdc`; the final layout was exported as `klayout/out/croc.gds`. Several backend-script changes were required because AMCP changed the memory configuration, floorplan, and timing/loading profile of the SoC.

The most important physical-design decision was to keep the external chip interface compatible with the Croc reference padframe. The AMCP therefore reuses existing GPIO pins instead of adding new chip-level pads. This avoids changing the pad-ring interface and keeps the project compatible with the provided bonding diagram. The final physical dimensions use the required 2500 μm by 2000 μm chip size including seal ring. Since the seal ring is added outside OpenROAD, the OpenROAD die area is 2416 μm by 1916 μm . The corresponding OpenROAD core area is defined inside the padframe and power-ring margin.

A second major change was the use of two larger SRAM macros. The original reference floorplan used two 512x32 SRAM macros. For this project, the SRAM configuration was increased to two 1024x64 macros to provide enough memory for non-trivial bare-metal firmware, AMCP tests, benchmarks, UART output, and closed-loop demonstration programs. This change required updating the OpenROAD floorplan script to use the `RM_IHPSG13_1P_1024x64_c2_bm_bist` macro dimensions. One subtle issue was that the SRAM macro master changed, but the synthesized instance suffix still contained the old generate-block name. Therefore the floorplan instance script had to be matched to the actual synthesized instance names instead of only to the new macro type. This was an OpenROAD instance-name issue, not a missing SRAM macro.

The SRAM placement also had to be adjusted. With the larger SRAMs placed too close to the top core boundary, the power-grid generation produced via-insertion warnings near the SRAM power grid. The SRAMs were therefore moved slightly downward and the macro halo was increased to 20 μm . This gave the SRAM grid and nearby standard-cell rows more spacing. The purpose of this change was not to optimize area, but to make the enlarged-memory floorplan compatible with the power grid and routing resources.

4.2 Placement and Timing-Repair Decisions

After floorplanning, placement was performed in two global-placement passes followed by detailed placement. A one-site global placement padding was added on the left and right of standard cells. This reduces local pin-access pressure and helps avoid full-GDS layout artifacts around dense standard-cell rows that may not be visible from the abstract LEF view alone.

After placement repair, some residual electrical-rule diagnostics remained. The automatic `repair_design` and `repair_timing` passes inserted buffers and resized cells to improve slew, capacitance, and timing. The final submitted implementation meets setup and hold timing, but some OpenROAD DRV warnings remain after finishing. These residual warnings are discussed separately in the signoff summary rather than being presented as timing failures.

4.3 Clock Tree Synthesis

The AMCP does not introduce a new clock domain. The PWM and QEN cores operate on the existing system clock, so backend timing closure remains a single main system-clock problem, together with the existing JTAG/ref-clock constraints from Croc.

A required CTS correction was the clock-pin name. The mapped EZ130 flip-flops use `CK`, not `CLK`, so the CTS script was updated to collect clock nets using `CK`. The final submitted CTS run also used `set_wire_rc -clock -layer Metal4`, obstruction-aware clock-tree synthesis, level balancing, a maximum sink-clustering diameter of 50, and an explicit `repair_clock_nets` step before setup repair. The final design meets setup and hold timing after routing and finishing, although OpenROAD still reports residual DRV diagnostics as discussed below.

Table 6: Final implementation summary.

System clock	100 MHz target; worst slack ≈ 1.13 ns
Setup / hold	0 setup violations, 0 hold violations
OpenROAD DRV diagnostics	71 max-slew, 71 max-cap, and 22 max-fanout warnings after finishing
DRC	Clean except 1024 known SRAM macro off-grid/min-grid markers
LVS	Pass
OpenROAD die / core area	$4.629 \text{ mm}^2 / 2.200 \text{ mm}^2$
Total / active area	$2.638 \text{ mm}^2 / 0.995 \text{ mm}^2$
Utilization	45.2 % core, 28.0 % standard-cell
Standard-cell logic	$467\,468 \mu\text{m}^2 \approx 82.8 \text{ kGE}$
SRAM macros	$2 \times 1024 \times 64$, total macro area 0.528 mm^2
AMCP area	$28\,372 \mu\text{m}^2 \approx 5.03 \text{ kGE}$
PWM / QEN area	$3799 \mu\text{m}^2 / 16\,493 \mu\text{m}^2$
Final post-layout power	67.8 mW with 15 pF IO-load model
VCD power	53.2 mW AMCP stress; 52.3 mW SW-PWM stress
IR drop	VDD 17.1 mV / 1.43 %; VSS 16.5 mV / 1.38 %

4.4 Routing, Antenna Repair, and Remaining Diagnostic Warnings

The routing stage was modified to make the final route more observable and conservative. As in the reference flow, signal and clock routing were restricted to Metal2–TopMetal1. In addition, the available routing capacity of Metal2–Metal3 and TopMetal1 was reduced. This discourages overuse of congested or coarse routing layers and leaves more routing flexibility for the power grid and detailed route.

After global routing, the script explicitly writes DRV reports before repair, after `repair_design`, after timing repair, after repaired global routing, and after final detailed routing. This was added because the default flow gives limited visibility into whether remaining warnings come from internal datapaths, pad loads, or antenna-cell insertion. The repair sequence was kept conservative: `repair_design` is followed by setup and hold repair with `-skip_pin_swap`, then inserted or resized cells are legalized and only modified nets are rerouted.

Antenna repair is performed before detailed routing using `repair_antennas -ratio_margin 30 -iterations 5`. The final detailed route is then run with `droute_end_iter 30`. This submitted flow therefore uses conservative global-route repair, antenna repair, and one final detailed-routing stage.

The final GDS-level DRC run is clean except for 1024 off-grid/min-grid markers located inside the provided SRAM macro layout. These markers originate from the supplied SRAM macro GDS and are not introduced by AMCP logic or by standard-cell routing. No project-specific DRC violations were observed outside this known SRAM macro issue. LVS also passes, confirming that the final extracted layout connectivity matches the generated LVS netlist.

OpenROAD still reports residual DRV diagnostics after finishing: 71 max-slew warnings, 71 max-cap warnings, and 22 max-fanout warnings. These are reported for transparency, but they are not GDS-level DRC or LVS failures. The max-slew and max-cap warnings are dominated by GPIO/output-pad paths under the 15 pF external-load assumption, while the fanout warnings are retained as OpenROAD implementation diagnostics. Since setup and hold timing are clean, LVS passes, and final GDS-level DRC is clean except for the known SRAM macro markers, these warnings were not treated as functional design failures.

4.5 Timing and Physical Signoff Summary

Table 6 summarizes both the final signoff result and the remaining OpenROAD diagnostic warnings. The distinction is important: setup/hold timing, LVS, and GDS-level DRC are clean, while the remaining OpenROAD DRV and antenna counts are retained as implementation diagnostics.

Gate equivalents were computed using the NAND2X1 cell area from the EZ130 TT Liberty file, $5.6448 \mu\text{m}^2$, as reference. Under this convention, the final standard-cell logic corresponds to approximately 82.8 kGE, while AMCP itself occupies approximately 5.03 kGE. SRAM macros and pads are reported separately because converting hard macros to NAND2-equivalent logic is less meaningful. Figure 6 shows the final routed layout after finishing.

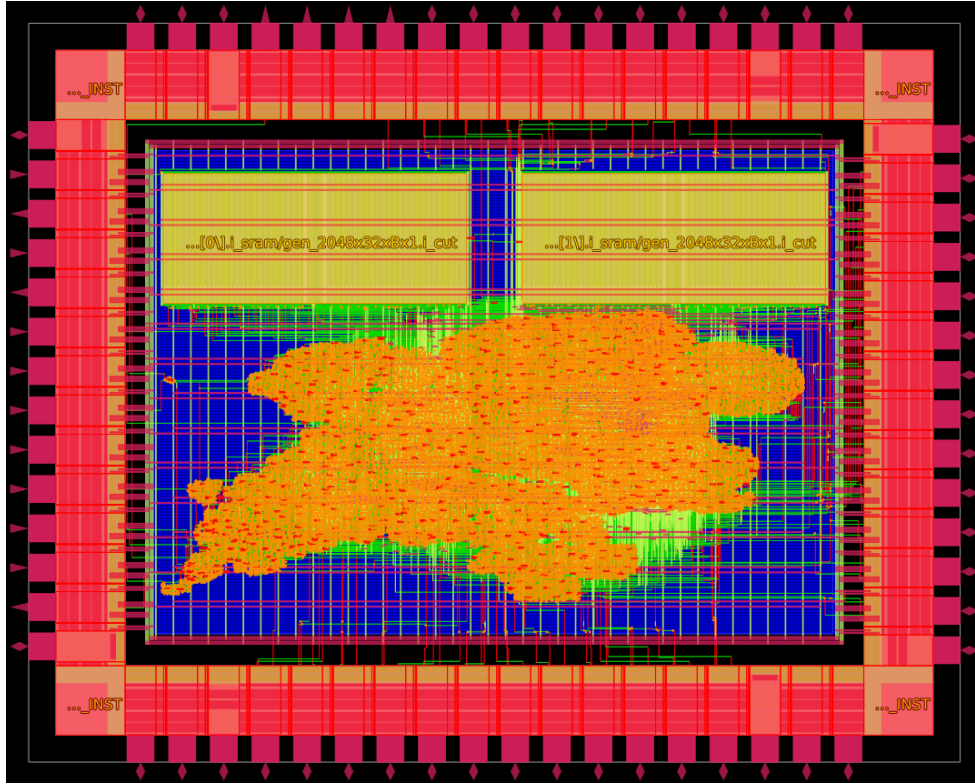


Figure 6: Final OpenROAD layout after placement, routing, antenna repair, and finishing. The pad ring surrounds the core, the two enlarged SRAM macros are placed near the top, and the AMCP/user logic is implemented as standard-cell logic inside the routed core region.

5 Power and IR-Drop Analysis

Power was evaluated using both statistical switching assumptions and workload-based VCD activity. The statistical sweep provides an activity bound: global activity factors of 0.01, 0.05, and 0.10 gave total powers of 41.5 mW, 63.5 mW, and 90.9 mW, respectively. For workload-based analysis, two firmware programs were simulated over a 400 μ s active window: a software-PWM stress workload and an AMCP hardware-control stress workload. The software workload consumed 52.3 mW, while the AMCP workload consumed 53.2 mW. The 0.9 mW difference corresponds to approximately 1.7 % full-chip overhead.

This should not be interpreted as an instantaneous power saving. The benefit of AMCP is architectural: PWM generation and encoder tracking are performed autonomously, so the CPU is not forced to spend cycles on continuous GPIO bit-banging or polling. With the 15 pF IO-load model, the final post-layout power is approximately 67.8 mW. Pads dominate this number at about 42.5 mW, or 62.8 %, followed by the clock network at about 16.5 mW, or 24.4 %. Sequential logic contributes about 7.01 mW, while macro and combinational power are below 1 mW each.

PDNSim reported all VDD and VSS shapes connected. The worst VDD drop is 17.1 mV, or 1.43 % of the 1.2 V supply. The worst VSS bounce is 16.5 mV, or 1.38 %. These values indicate that the enlarged SRAM configuration and AMCP logic did not create an excessive IR-drop issue.

6 Conclusion

This project extends Croc with AMCP, a memory-mapped motor- and power-control peripheral for deterministic embedded-control I/O. AMCP adds hardware PWM generation, quadrature encoder tracking, GPIO alternate-function routing, and software-visible control/status registers while preserving the existing Croc padframe and test infrastructure.

Functional tests verify AMCP register access, PWM operation, QEN simulation-input mode, external GPIO QEN mode, PWM/QEN offload benchmarks, and closed-loop control-style demonstrations. Backend implementation required floorplan changes for larger SRAMs, corrected SRAM macro instance mapping, clock-pin-name correction for the mapped library, CTS tuning, conservative routing repair, antenna repair, and final detailed routing.

The final design meets timing with zero setup and hold violations, passes LVS, and has clean DRC except for the known SRAM macro off-grid/min-grid markers. Post-layout analysis shows that AMCP adds deterministic control capability with small area and full-chip power overhead, while IR drop remains below 1.5 % of the 1.2 V supply.

THE END.

Repository. <https://github.com/Rishabh-Barola/CrocDrive.git>

References

- [1] P. Sauter, T. Benz, P. Scheffler, H. Pochert, L. Wüthrich, M. Povišer, B. Muheim, F. K. Gürkaynak, and L. Benini, “Croc: An end-to-end open-source extensible risc-v mcu platform to democratize silicon,” 2025.
- [2] OpenHW Group, “Cv32e20: An embedded 32-bit risc-v cpu core,” <https://docs.openhwgroup.org/projects/cve2-user-manual/en/latest/>, accessed: 2026-06-30.
- [3] PULP Platform, “Open bus interface systemverilog ips,” <https://github.com/pulp-platform/obi>, accessed: 2026-06-30.
- [4] M. Hagen and V. Yousefzadeh, “Applying digital technology to pwm control-loop designs,” in *Texas Instruments Power Supply Design Seminar*, 2008, https://www.ti.com/download/trng/docs/seminar/Topic_7_Hagen.pdf.
- [5] Dynapar, “Quadrature encoders,” <https://www.dynapar.com/knowledge/encoder-basics/encoder-output/quadrature-encoders/>, accessed: 2026-06-30.